

Centro Regional Universitario Córdoba IUA

Maestría en Ciberdefensa



Trabajo Práctico N°2

Generador de números aleatorios de Java

Módulo:

- Criptografía

Alumno:

- Ing. Vietto Herrera Santiago

Docentes:

- Ing. Montes Miguel

14 de Mayo de 2026

Córdoba - Argentina

Indice

Desarrollo	1
1. Generadores de números pseudoaleatorios	1
1.1. Introducción	1
1.2. Lenguaje Java	1
2. Solución del desafío	2
2.1. Análisis del problema	2
2.2. Implementación de la solución	6
2.2.1 Conexión con servidor y solicitud de dos valores aleatorios consecutivos	6
2.2.2 Conexión con servidor y solicitud de dos valores consecutivos	7
2.2.3 Obtención del estado del segundo valor	7
2.2.4 Predicción del próximo número	9
2.2.5 Envío de la respuesta al servidor	9

Desarrollo

1. Generadores de números pseudoaleatorios

1.1. Introducción

Un generador de números pseudoaleatorios o PRNG (Pseudo Random Number Generator) produce una secuencia de números estadísticamente aleatorios.

- No son realmente aleatorios, pero lo parecen, lo cual es suficiente para muchas aplicaciones computacionales. Por ejemplo, si se quiere hacer una simulación, usar el método montecarlo para simular algo, programar un juego, etc, nos alcanza con ese nivel de aleatoriedad y estadística.
- Se inicializan con una semilla (seed) o valor inicial, y a partir de allí generan una secuencia de números que satisfacen ciertas propiedades estadísticas. Pero dada la misma semilla, siempre producen la misma secuencia de números, es decir, producen un resultado que es predecible.
- La secuencia producida por una semilla dada es determinística.
- Tienen un estado interno finito, por lo que tienen ciclos (a partir de cierto momento la secuencia se repite). Dicho de otra forma, si tenemos una función en el que cada vez que se invoca, nos devuelve un valor diferente, es porque tiene un estado interno que no vemos, el cual tiene un tamaño finito que en algún momento se va a repetir, y si el estado se repite, la secuencia que se va a generar a partir de esa repetición del estado la misma, y por lo tanto se va a tener un ciclo. A partir de un cierto momento, los valores que generamos se repiten

1.2. Lenguaje Java

El lenguaje Java utiliza un generador congruencial lineal. Estos generadores tienen una fórmula con las siguientes características:

$$X_{i+1} = (AX_i + C) \text{ mód } M$$

Básicamente esto significa que el siguiente valor que se va a producir es el valor anterior que se produjo multiplicado por una constante, sumado por otra constante, y finalmente se calcula el resto de todo lo anterior dividido por otra constante. Estas tres constantes están definidas para el algoritmo. En este caso particular, se utiliza la siguiente fórmula la cual actualiza el estado interno del generador:

$$\text{seed} = (\text{seed} * 0x5DEECE66DL + 0xB) \& ((1L \ll 48) - 1)$$

En donde podemos observar:

- Constante A: 0x5DEECE66DL
- Constante C: 0xB (11 en decimal)
- Módulo: 2^{48} . Como es una potencia de 2, puede ser optimizado y en lugar de calcular el módulo, se utiliza el operador AND con la secuencia de 48 unos. Básicamente es equivalente a calcular el resto de dividir por 2^{48} .

Como el estado interno es de 48 bits, por lo tanto el máximo ciclo posible es 2^{48} , y llegado al límite se genera la misma secuencia.

Produce enteros con la cantidad de bits pedida, hasta 32 bits, devolviendo:

```
(int)(seed >>> (48 - bits))
```

De los 48 bits, toma los bits más significativos. Esta operación es el desplazamiento hacia la derecha de los bits sin signo.

Y para devolver un número de punto flotante o double, lo que hace es pedir dos veces, en donde en el primer llamado saca 26 bits, en el siguiente 27, luego los une y los divide por 2^{53} para obtener un valor entre 0 y 1. Es 2^{53} , porque en un número de punto flotante de 64 bits, la cantidad de valores que tenemos entre el 0 y el 1 son 2^{53} valores posibles.

Esto no es criptográficamente seguro.

2. Solución del desafío

2.1. Análisis del problema

El presente trabajo práctico tiene como objetivo explotar una debilidad del PRNG de Java, específicamente el generador compatible con `java.util.Random`. En este caso el servidor de la IUA tiene un generador pseudoaleatorio compatible con Java, en la que por medio de un GET a una URL, utilizando el correo electrónico registrado del alumno que lleva a cabo la actividad, el servidor devuelve un entero de 32 bits con signo. En donde lo que se busca es adivinar el siguiente número de 32 bits que produciría ese mismo generador y enviarlo mediante un POST al servidor para determinar si es correcto o no.

El problema existe porque `java.util.Random` no genera números realmente aleatorios como vimos anteriormente. Si conocemos el estado del generador, es decir el valor de seed, es muy fácil saber cuál es el número siguiente. Pero al pedir un entero, no obtenemos el estado del generador, sino solo 32 bits de ese estado. Nos faltan 16 bits. Sin embargo, si tenemos la salida de dos números del generador, podemos probar todos los valores posibles de esos 16 bits en el primer número y verificar cuál produce un estado capaz de generar el segundo número.

Por eso, si el servidor nos devuelve un número, conocemos los 32 bits altos del estado, pero nos faltan los 16 bits. Como faltan solo 16 bits, se pueden probar todas las posibilidades: $2^{16} = 65536$ posibilidades, lo cual es muy poco para una computadora.

Entonces, lo que se busca hacer es:

1. Pedir un primer número al servidor.
2. Pedir un segundo número al servidor.
3. Usar el primer número para reconstruir todos los posibles estados internos.
4. Avanzar cada estado candidato una vez.
5. Comparar cuál produce el segundo número.
6. Una vez encontrado el estado correcto, avanzar una vez más.
7. Ese nuevo valor será la respuesta que hay que enviar al servidor.

Con un solo número no alcanza para saber exactamente el estado interno, porque faltan 16 bits. Ya que si el servidor devuelve valor_1, entonces el estado interno después de generar esa salida tiene esta forma:

- valor_1 || 16 bits desconocidos

Esos 16 bits desconocidos pueden ser:

```
0000000000000000
0000000000000001
0000000000000010
...
1111111111111111
```

O sea, hay 65536 posibles estados. Entonces, se solicita un segundo número valor_2, y ahora se prueba cada candidato del primer estado, en donde el candidato correcto será aquel que, al avanzar el PRNG una vez, produzca exactamente valor_2. Cuando encontramos ese candidato, ya recuperamos el estado interno real del generador.

Para lograr una mejor comprensión se utiliza un ejemplo. Se supone que Java generó el siguiente número -1210225942. Este está expresado como entero de 32 bits con signo. Los enteros de 32 bits pueden interpretarse de dos formas:

- Con signo: -1210225942
- Sin signo: 3084741354

Para reconstruir el estado interno del generador no nos interesa tanto el valor “matemático con signo”, sino sus 32 bits exactos. Esos mismos 32 bits, interpretados como entero sin signo, valen 3084741354.

- $3084741354 = -1210225942 + 2^{32}$

Respecto al estado interno, cuando Java hace nextInt(), no devuelve el seed completo. El seed tiene 48 bits del estado interno, pero nextInt() devuelve solo los 32 bits superiores:

- [32 bits visibles][16 bits ocultos]

Entonces, si vemos este número 3084741354, sabemos que los 32 bits superiores del estado eran esos. Es decir, el estado interno tenía esta forma:

- 3084741354 | ??????

Donde ?????? representa los 16 bits inferiores que no conocemos. Entonces, como faltan 16 bits, hay $2^{16} = 65536$ posibilidades, o sea, el estado interno podría ser:

```
3084741354 | 0000000000000000
3084741354 | 0000000000000001
3084741354 | 0000000000000010
...
3084741354 | 1111111111111111
```

En decimal sería algo así:

- $3084741354 \times 2^{16} + \text{valor_desconocido}$

Entonces el razonamiento sería, que el número observado tiene 32 bits del estado, pero hay que descubrir los 16 bits restantes.

Ahora, suponemos que el siguiente número generado por Java fue -1908238811. Este está expresado como entero de 32 bits con signo. Nuevamente, los enteros de 32 bits pueden interpretarse de dos formas:

- Con signo: -1908238811
- Sin signo: 2386728485

Entonces, tenemos dos salidas consecutivas:

- Primera salida: -1210225942
- Segunda salida: -1908238811

Como son enteros de 32 bits con signo, para trabajar con los bits los interpretamos como enteros sin signo:

- Primera salida visible: 3084741354
- Segunda salida visible: 2386728485

Todavía no sabemos cuál de los 65536 estados posibles era el real. Pero tenemos una pista, el segundo número generado. Entonces se razona que, si se prueba un posible estado y se lo avanza una vez con la fórmula de Java, el estado correcto debería producir como salida el segundo número (en salida visible).

- $\text{nuevo_seed} = (\text{seed} \times 25214903917 + 11) \bmod 2^{48}$

Y la prueba sería algo como:

```
estado = 3084741354 × 216 + 0 ; el estado resultado no produce 2386728485
estado = 3084741354 × 216 + 1 ; el estado resultado no produce 2386728485
estado = 3084741354 × 216 + 2 ; el estado resultado no produce 2386728485
...
estado = 3084741354 × 216 + 65347 ; el estado resultado si produce 2386728485
```

En este ejemplo, el valor oculto es 65347. Entonces, el estado del primer número es:

- $\text{estado} = 3084741354 \times 2^{16} + 65374 = 202161609441118$

Siendo:

- 32 bits superiores: 3084741354
- 16 bits inferiores: 65374

Lo importante es que nosotros al principio no sabemos que los 16 bits ocultos valen 65374. Pero podemos probar todos los valores desde 0 hasta 65535, y quedarnos con el único que, al avanzar el generador, produce el segundo número observado.

Una vez recuperado el estado de la seed actual, predecimos el número siguiente. Después de validar con el segundo número, ya conocemos el estado real del generador. Entonces avanzamos una vez más la fórmula:

- $\text{siguiente_seed} = (\text{seed_actual} \times 25214903917 + 11) \bmod 2^{48}$

Siendo para este caso la fórmula reemplazada por los datos obtenidos, en la que encontramos el seed_2 del segundo número entregado por Java.

- $(202161609441118 \times 25214903917 + 11) \bmod 2^{48} = 156416638005009$

Tomando los 32 bits superiores del seed_2 $\gg 16$ obtenemos 2386728485 justamente.

Y por último, como no necesitamos conocer de antemano un tercer número entregado por Java, lo que hacemos es utilizar el seed_2 para conocer el seed_3.

- $\text{seed}_3 = (\text{seed}_2 \times 25214903917 + 11) \bmod 2^{48}$

Siendo así:

- $(156416638005009 \times 25214903917 + 11) \bmod 2^{48} = 274519139438664$

Tomando los 32 bits superiores del seed_3 $\gg 16$ obtenemos 4188829642 justamente, el cual está como unsigned de 32 bits. Y como es mayor que 2^{31} , Java lo interpreta como negativo, dado que:

- $4188829642 > 2^{31} (2147483648)$

Entonces, hay que convertirlo a entero con signo restando:

- $4188829642 - 2^{32} (4294967296) = -106137654$

Determinando entonces que el próximo número es -106137654.

2.2. Implementación de la solución

2.2.1 Conexión con servidor y solicitud de dos valores aleatorios consecutivos

Dado a lo entendido en la sección 2.1, como primer paso tenemos la solicitud de números aleatorios al servidor:

1. Se definen las credenciales de acceso al servidor, que son por un lado el correo electrónico registrado del alumno que lleva a cabo el siguiente trabajo práctico, y la URL del servidor.

```
6
7 server = "https://cripto.iua.edu.ar"
8 email = "santiagovietto5@gmail.com"
9
```

2. Luego, se declara una función `get_challenge()` para hacer posible la comunicación con el servidor y solicitar a este mediante un GET, valores o números aleatorios. Esta convierte el texto recibido a variable de tipo `int` (número entero), para trabajar.

```
34 def get_challenge():
35     response = requests.get(f"{server}/javarand/{email}/challenge")
36     response.raise_for_status()
37     return int(response.text)
```

3. Como se requieren dos números aleatorios, se declaran dos variables diferentes, las cuales llaman a la función `get_challenge()`, y almacenan el valor obtenido de la solicitud. Luego se muestran en pantalla los valores con signo obtenidos:

```
51
52 value_1 = get_challenge()
53 value_2 = get_challenge()
54
55 print("\nPrimer valor entregado por Java:", value_1)
56 print("Segundo valor entregado por Java:", value_2)
57
```

```
• (venv_TPsCrypto) santiago-vietto@santiago-vietto-pc:~/Documents/Trabajos-Practicos-Criptografia$ python
Primer valor entregado por Java: -65015977
Segundo valor entregado por Java: -1711134734
```

Como sabemos, el primer número nos permite saber los 32 bits superiores del seed, pero faltan 16 bits. Y el segundo número sirve para comprobar cuál de las 65536 posibilidades es la correcta.

2.2.2 Conexión con servidor y solicitud de dos valores consecutivos

Como segundo paso, se convierten los dos valores obtenidos a unsigned de 32 bits, y esto se hace para poder trabajar con los bits.

1. Como primera medida se define una función `to_unsigned_32()` para convertir un número entero de 32 bits con signo a su interpretación sin signo. Nuevamente, el servidor devuelve números enteros de 32 bits con signo, lo que significa que además puede devolver valores negativos. Pero para trabajar con bits nos conviene interpretarlo como unsigned.

La operación consiste en tomar la representación binaria del número, y al aplicar la máscara `0xffffffff` se conserva solo los 32 bits inferiores, para evitar que se lo trate como negativo.

```
25
26 def to_unsigned_32(value):
27     return value & 0xffffffff
28
```

2. Luego se crean dos variables las cuales hacen llamado de la función `to_unsigned_32()`, pasándole por parámetros los valores obtenidos en el paso anterior respectivamente. Y finalmente mostramos los nuevos valores obtenidos:

```
61
62 value_1_unsigned = to_unsigned_32(value_1)
63 value_2_unsigned = to_unsigned_32(value_2)
64
65 print("\nPrimer valor sin signo:", value_1_unsigned)
66 print("Segundo valor sin signo:", value_2_unsigned)
67
```

```
Primer valor sin signo: 4229951319
Segundo valor sin signo: 2583832562
```

2.2.3 Obtención del estado del segundo valor

Esta parte es el núcleo del ataque, ya que se busca reconstruir el estado interno del generador de Java probando los 16 bits que no conocemos.

1. Se inicializa el estado como desconocido, ya que todavía no encontramos el estado interno correcto, y se inicializa en `None` para después poder verificar si se logró encontrar un candidato válido.

2. En el ciclo `for`, se prueban todos los valores posibles para los 16 bits inferiores, en donde $1 \ll 16$ equivale a $2^{16} = 65536$, recorriendo del 0 al 65535.

2.1. Se construye un posible seed completo, en donde se arma un candidato de seed de 48 bits. Sabemos que `value_1_unsigned` son los 32 bits superiores del estado. Entonces, `value_1_unsigned << 16` mueve esos 32 bits hacia la izquierda, dejando lugar para los 16 bits inferiores. Y `low_bits` rellena estos últimos 16 bits con una posibilidad. Por ejemplo, si tenemos lo siguiente dado a los resultados obtenidos:

- `value_1_unsigned = 4229951319`
- `low_bits = 65374`

Se forma:

- $\text{candidate_seed} = 4229951319 \times 2^{16} + 65374$

2.2. Ahora, se toma ese posible seed y se lo avanza una vez con la fórmula de Java, la cual está declarada previamente en la función `next_seed()`.

```
29
30 def next_seed(seed):
31     return (seed * MULTIPLIER + ADDEND) & MASK
32
```

En donde recibe como parámetro el estado candidato y aplica la fórmula:

- $\text{nuevo_seed} = (\text{seed} \times 25214903917 + 11) \bmod 2^{48}$

2.3. Como resultado de `next_candidate_seed` se debería obtener el `seed_2` completo correspondiente al `value_2`. Pero como Java no devuelve el seed completo, se remueven los 16 bits inferiores del `seed_2`, quedándonos con los 32 bits superiores. Por lo que el valor de `candidate_output` es el número unsigned que ese `seed_1` candidato produciría como segunda salida.

2.4. Entonces, se compara si un candidato `seed_1`, al avanzar una vez, produce exactamente el segundo número unsigned que proporcionó el servidor, entonces se encontró el estado siguiente o `seed_2` correcto y se almacena en la variable `state_after_first`. Si la respuesta es no, ese candidato se descarta automáticamente.

3. Finalmente, se muestra por pantalla el `seed_2` obtenido necesario para la predicción.

```
72 state_after_first = None
73
74 for low_bits in range(1 << 16):
75     candidate_seed = (value_1_unsigned << 16) | low_bits
76
77     next_candidate_seed = next_seed(candidate_seed)
78     candidate_output = next_candidate_seed >> 16
79
80     if candidate_output == value_2_unsigned:
81         state_after_first = next_candidate_seed
82         break
83
84 if state_after_first is None:
85     raise Exception("No se pudo recuperar el estado interno.")
86
87 print("\nEstado/semilla recuperada del segundo valor:", state_after_first)
88
```

```
Estado/semilla recuperada del segundo valor: 169334050845918
```

2.2.4 Predicción del próximo número

En este paso, finalmente se predice el próximo número que va a devolver Java.

1. Se aplica nuevamente la fórmula del generador de Java, con la función `next_seed()`, pasando como parametro `state_after_first`, ya que es el estado correspondiente o `seed_2` del segundo número proporcionado, y al avanzar obtenemos el estado correspondiente al próximo número, es decir, un `seed_3`.
2. Una vez obtenido el `seed_3`, se desplazan los 16 bits inferiores para quedarnos con los 32 superiores del `seed_3`. Nuevamente, esto se hace porque Java no devuelve el seed completo de 48 bits, sino que cuando genera un int, toma los 32 bits superiores del seed.
3. Como el valor obtenido es unsigned, se declara previamente una función `to_signed_32()`, la cual compara si el valor unsigned es mayor o igual a $2^{31} = 2147483648$, en donde si es así, significa que en Java se interpreta como negativo, y se le resta $2^{32} = 4294967296$. Entonces esta función sirve para que la respuesta final tenga el mismo formato que espera el servidor, es decir, entero decimal con signo.

```
34 def to_signed_32(value):
35     if value ≥ 2**31:
36         return value - 2**32
37     return value
38
```

4. Finalmente, una vez realizada la conversión, se muestra el valor del número que se logró predecir.

```
92
93 next_state = next_seed(state_after_first)
94 predicted_unsigned = next_state >> 16
95 predicted_signed = to_signed_32(predicted_unsigned)
96
97 print("\nPredicción:", predicted_signed)
98
```

```
Predicción: -814249191
```

2.2.5 Envío de la respuesta al servidor

Una vez que se predijo el siguiente número, el objetivo ahora es enviarlo al servidor para que valide si el desafío fue resuelto correctamente.

- 1) Se crea la URL que apunta al endpoint donde el servidor espera recibir el mensaje descifrado.
- 2) Luego, se realiza una petición POST en la que se envía el número predicho usando el campo de formulario llamado `number`. Se convierte el valor a string porque http no envía enteros de Python directamente. Y se convierte el texto a bytes ASCII, porque es el formato que espera requests cuando se usa dentro de files.

```

95 response_answer = requests.post(
96     f"{server}/javarand/{email}/answer",
97     files={
98         "number": str(predicted_value_signed).encode("ascii")
99     }
100 )
101
102 print("\nRespuesta del servidor:")
103 print(f"Status code: {response_answer.status_code}")
104 print(response_answer.text)
105

```

3) Finalmente, se imprime la respuesta del servidor, lo que permite ver si el envío fue correcto, mostrando el código de estado de la petición http al servidor, por ejemplo, código 200 como en este caso que salió bien, y el contenido de `response_answer.text` que muestra el mensaje que devuelve el servidor, en este caso ¡Ganaste!, indicando que la actividad fue resuelta correctamente.

```

Respuesta del servidor:
Status code: 200
¡Ganaste!

```